

1. Introduction

This document describes the design, retrieval architecture, and evaluation methodology of the **NexaForge Manufacturing Equipment Maintenance Query Agent** — a retrieval-augmented generation (RAG) system built to answer natural-language questions from equipment manuals, maintenance schedules, and troubleshooting guides for factory floor and maintenance teams.

Manufacturing floor workers frequently need quick answers from lengthy equipment manuals during time-critical repairs. Manually searching PDF documents is slow and error-prone, especially when the required information is buried inside specification tables or multi-page troubleshooting sections. The system described here ingests manuals automatically, indexes them for hybrid semantic and keyword search, and answers operator questions with citations to the exact source page.

1.1 System Overview

The assistant is exposed through a floating chat widget available on both the Admin and Worker dashboards. Every response is generated strictly from ingested manual content and includes the source document name and page number, so operators can verify the answer against the original manual if needed.

2. Document Ingestion Pipeline

Uploaded documents (PDF, TXT, PNG, or JPG) are processed by a background task with four stages: text extraction, chunking, embedding, and storage.

2.1 Text and Table Extraction

PDF text is extracted using pdfplumber. Specification and maintenance-schedule tables are extracted separately using table-structure detection and rewritten as explicit “label: value” lines, which prevents column values from being lost when a table is flattened into plain text. Pages using a two-column academic layout are detected by checking for a wide empty vertical gutter near the horizontal center of the page; when detected, the left column is read completely before the right column, avoiding the interleaved reading order that a naive top-to-bottom text extraction would otherwise produce.

2.2 Chunking and Embedding

Extracted text is split into chunks of approximately 1200 characters with 200 characters of overlap using a recursive character splitter. Table content is chunked separately from surrounding prose to avoid a specification row being split across a chunk boundary. Each chunk is embedded using a

remote sentence-transformer model (384-dimensional vectors) and stored in a PostgreSQL database with the pgvector extension.

3. Hybrid Retrieval and Reranking

Given a query, the system runs two retrieval methods in parallel and merges their results before a final reranking pass.

3.1 Keyword and Semantic Search

Keyword search uses the BM25 Okapi algorithm over all stored chunks. Semantic search computes cosine similarity between the query embedding and each chunk embedding using pgvector’s nearest-neighbor search. The two result sets are normalized to a common 0–1 scale and merged using a weighted average, with both methods weighted equally by default.

3.2 LLM-Based Reranking

The top candidates from hybrid search are re-scored by a large language model, which reads each candidate passage alongside the original question and returns the passages ordered from most to least relevant. This step was previously implemented with a hosted cross-encoder model, but was replaced with an LLM-based approach after the hosted model became unavailable through the inference provider being used. The reranker includes a fallback: if the reranking call fails for any reason, the system reverts to the hybrid search ranking rather than returning an error.

4. Conversational Agent Design

Each incoming question is classified into one of four categories by a routing model before any retrieval occurs.

Category	Description
search_manual	Troubleshooting, error codes, procedures
check_schedule	Maintenance intervals and scheduled tasks
log_issue	Reporting a newly observed problem
escalate	Unclear, unsafe, or out-of-scope questions

Table 1. Query routing categories used by the agent.

Questions routed to search_manual or check_schedule retrieve up to five reranked chunks, which are passed to a language model along with an instruction to answer strictly from the provided context and cite the page number. If the retrieved context does not contain the answer, the model is instructed to state this rather than speculate. Questions routed to log_issue are recorded for later review. Questions routed to escalate return a message directing the operator to

a qualified technician.

5. Evaluation Methodology

A fixed evaluation set of 25 questions was constructed, covering troubleshooting, scheduled maintenance, cross-document production-flow questions, and deliberately out-of-scope questions expected to trigger escalation. Two metrics are reported for each evaluation run.

5.1 Escalation Accuracy

Escalation accuracy measures whether the routing model made the correct decision to answer or escalate each question, compared against a manually labeled ground truth. This metric does not evaluate whether the resulting answer content was correct.

5.2 Answer Accuracy

Answer accuracy checks whether the generated answer text contains a set of required keywords defined per question — for example, a spindle speed range question requires both bound values to be present in the answer. This metric specifically catches cases where the routing decision was correct but retrieval failed to surface the correct passage, resulting in an incorrect “not found” response.

6. Results

Prior to fixing specification-table extraction, the system scored 63.6% on answer accuracy, with all observed failures corresponding to specification-table lookups such as spindle speed range and tool changer capacity. After implementing table-aware extraction and table-isolated chunking, answer accuracy improved to 90.9% on the same evaluation set, while escalation accuracy remained at 100% throughout, indicating that the routing model’s category decisions were not the source of the original failures.

The one remaining failure at the time of writing involved two passages within the same manual that both referenced spindle RPM values — one from the specification table and one from an unrelated warm-up procedure — indicating a retrieval disambiguation issue rather than a missing-data issue.

7. Conclusion and Future Work

The evaluation results demonstrate that table-aware extraction has a measurable, significant effect on answer quality for manuals containing specification tables, independent of the routing and generation stages of the pipeline. Future work includes disambiguating near-duplicate numeric references within a single document,

extending column-layout detection to mixed single/multi-column pages, and expanding the evaluation set as additional equipment manuals are ingested.